Ejercicios de aplicación 🤓

L3: Elementos básicos de Spark

Es hora de que pongas en práctica todo lo aprendido. 🤓
Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que, a continuación, encontrarás diferentes desafíos que podrás resolver de forma independiente y a tu ritmo.

Más adelante, conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

Construir un flujo de procesamiento en PySpark que cargue datos en un RDD, aplique transformaciones clave (map, filter, flatMap, distinct, sortBy) y ejecutar acciones (take, collect, count, sum/mean/stdev), mostrando además el linaje y la ejecución de un Job.

Consigna resumida

- ➔ *Inicializa Spark y crea un RDD a partir de un archivo de texto.*
- ➔ *Genera un Pair-RDD palabra→1 y calcula frecuencias.*
- ➔ *Aplica transformaciones: filter, map, flatMap, distinct, sortBy.*
- ➔ *Ejecuta acciones: count, take, collect, takeSample, sum/mean/stdev (según corresponda).*
- ➔ *Muestra linaje (toDebugString) y explica brevemente qué Job Spark se dispara al invocar una acción.*
- ➔ *Persiste el RDD clave y compara tiempos con/sin caché.*

2. **¿Dónde se lleva a cabo? 👩‍💻**

   Visual Studio Code

3. **Tiempo de dedicación ⏳**

2 horas

4. **Recursos** 🛠️

   RDDAPI: https://spark.apache.org/docs/latest/rdd-programming-guide.html

5. **Plus** ➕

   Particionar manualmente el RDD y medir impacto (repartition/coalesce).

6. ⚠️**Condición**

   Esta práctica o ejercitación no requiere ser entregada o evaluada por el mentor. No obstante, puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 👇

L3: Elementos básicos de Spark

➔ Inicialización del contexto de Spark

```
# Si estás en Colab:
!pip -q install pyspark

from pyspark import SparkContext
sc = SparkContext(master="local[*]", appName="RDD-Basics")
sc.setLogLevel("WARN")
```

➔ Carga de datos en un RDD

```
# Sube un archivo .txt a Colab o usa una ruta local
ruta = "/content/shakespeare.txt"  # ajusta según tu entorno
text_rdd = sc.textFile(ruta)  # Cada elemento = una línea
print("Lineas:", text_rdd.count())
print("Primeras líneas:", text_rdd.take(3))
```

➔ Transformaciones fundamentales

```
# flatMap: dividir en palabras; map: normalizar; filter: descartar vacías/no alfabéticas
words_rdd = (text_rdd
             .flatMap(lambda line: line.split())
             .map(lambda w: w.lower().strip(",.;:!?\"'()[]"))
             .filter(lambda w: len(w) > 0))

# distinct: cuántas palabras únicas
unique_words = words_rdd.distinct().count()
print("Palabras únicas:", unique_words)

# Pair-RDD (palabra, 1), agregación por clave y orden
pairs_rdd = words_rdd.map(lambda w: (w, 1))
freq_rdd  = pairs_rdd.reduceByKey(lambda a, b: a + b)

# sortBy: ordenar por frecuencia descendente
top20 = freq_rdd.sortBy(lambda kv: kv[1], ascending=False).take(20)
print("Top 20 palabras:", top20[:10], "...")  # mostrar parte
```

➔ Acciones típicas

```
total_palabras = words_rdd.count()              # acción
muestra        = words_rdd.takeSample(False, 10)
print("Total palabras:", total_palabras)
print("Muestra:", muestra)

# Para estadísticas numéricas, proyectamos solo las frecuencias (valores):
freq_values_rdd = freq_rdd.map(lambda kv: kv[1])
print("Sum:", freq_values_rdd.sum())
print("Mean:", freq_values_rdd.mean())
print("Stdev:", freq_values_rdd.stdev())
```

➔ Linaje y Lazy Evaluation

```
# Linaje del RDD de frecuencias
print(freq_rdd.toDebugString())
```

Breve explicación: las transformaciones (flatMap, map, filter, distinct, reduceByKey, sortBy) se registran en el linaje (DAG), pero no se ejecutan hasta que llamamos a una acción como count(), take(), sum(), etc. En ese momento, Spark construye el plan, lo divide en *stages* y *tasks*, y lanza un *Job* para devolver el resultado al *driver*.

➔ Persistencia/caché y comparación

```
# Persistir el RDD intermedio que reutilizarás varias veces
from pyspark import StorageLevel

cached_words = words_rdd.persist(StorageLevel.MEMORY_ONLY)
sc.setLogLevel("INFO")

# Primera acción: materializa el caché
_ = cached_words.count()

# Reutiliza el RDD varias veces y observa tiempos aproximados
import time
t0 = time.time(); _ = cached_words.distinct().count(); t1 = time.time()
t2 = time.time(); _ = cached_words.sample(False, 0.2).count(); t3 = time.time()

print(f"Distinct (cached)   : {t1 - t0:.3f}s")
print(f"Sample 20% (cached) : {t3 - t2:.3f}s")

# Comparar sin cache (opcional):
uncached_words = words_rdd.unpersist().cache()  # técnica para limpiar/recachear
```

➔ Resultados esperados

- Conteo de líneas y palabras mostradas correctamente.
- Top 20 palabras con frecuencias ordenadas descendentemente.
- Estadísticas (sum/mean/stdev) sobre frecuencias numéricas.
- toDebugString() imprime el linaje con transformaciones y particiones.
- Con cache()/persist() verás reducción de tiempo en operaciones repetidas.

➔ Cierre técnico

Creaste y manipulaste RDDs, aplicando transformaciones y acciones, comprobando *lazy evaluation*, linaje, persistencia y observando cómo se dispara al menos un Job Spark en cada acción. Este flujo cubre los criterios del módulo: creación de RDD, transformaciones/acciones y uso correcto del entorno Spark para un problema de procesamiento masivo de baja complejidad.